

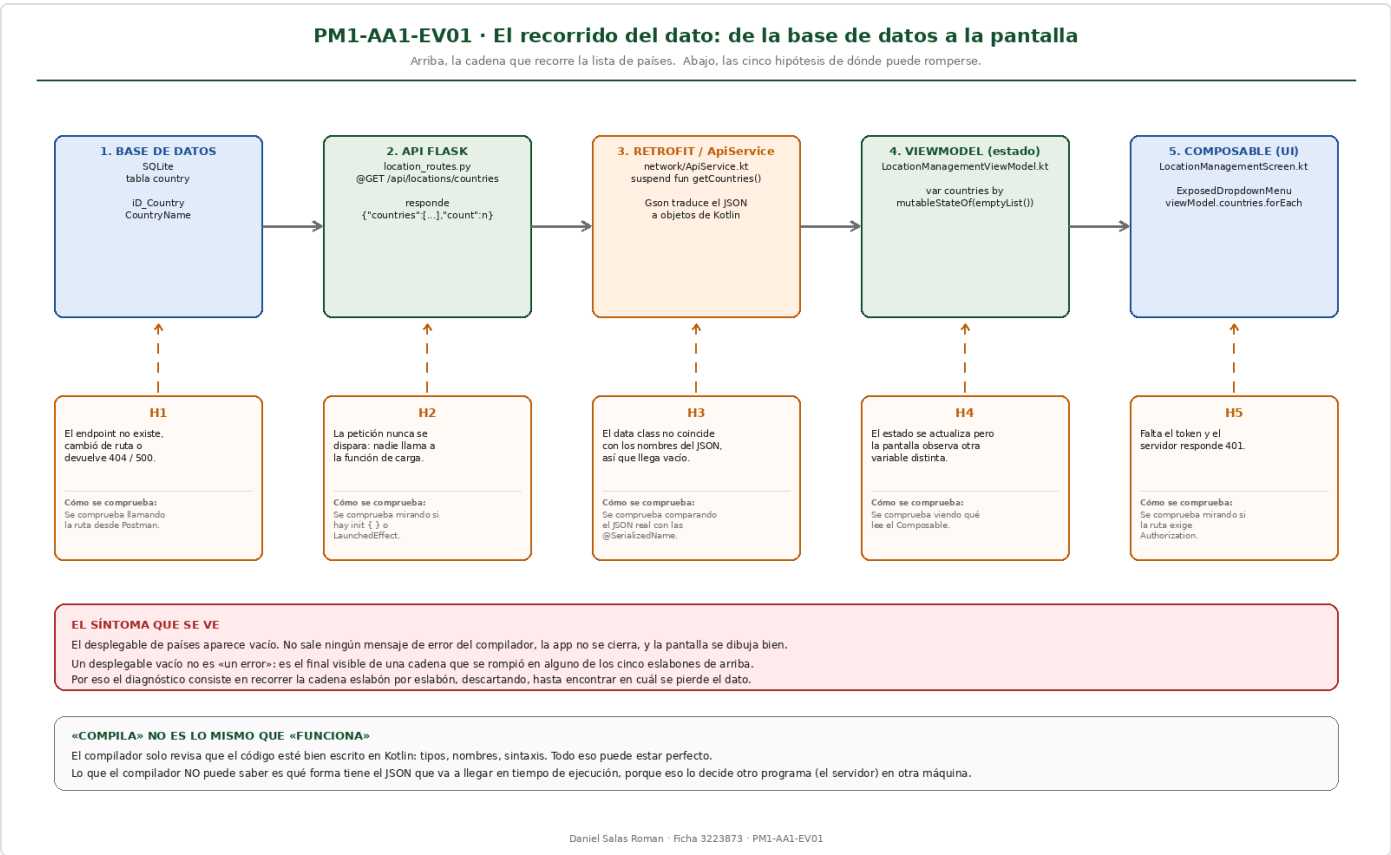
PM1-AA1-EV01 — Reflexión: ¿por qué falla un desplegable vacío?

Plan de Mejoramiento · Depuración y complemento de la app móvil de e-commerce

APRENDIZ	FICHA	PROGRAMA	FECHA
Daniel Salas Roman	3223873	Análisis y Desarrollo de Software (228118)	

1. El recorrido del dato

Éste es el camino que debería seguir la lista de países desde la base de datos hasta la pantalla:



#	ESLABÓN	ARCHIVO	QUÉ HACE
4	ViewModel (estado)	<code>LocationManagementViewModel.kt</code>	Guarda la lista recibida en una variable observable
5	Composable (UI)	<code>LocationManagementScreen.kt</code>	Lee ese estado y dibuja el menú desplegable

Si cualquiera de esos cinco eslabones falla, el resultado visible es el mismo: un desplegable vacío. Por eso el síntoma no dice nada por sí solo, y el diagnóstico consiste en recorrer la cadena descartando eslabones hasta encontrar dónde se pierde el dato.

2. Las hipótesis

La guía pide al menos cuatro. Planteo cinco, cada una con la forma concreta de comprobarla, porque una hipótesis que no se puede comprobar no sirve para depurar.

	HIPÓTESIS	CÓMO LA COMPRUEBO	QUÉ VERÍA SI FUERA CIERTA
H1	El endpoint no existe, cambió de ruta o devuelve un error (404 / 500).	Llamo la ruta directamente desde Postman, sin pasar por la app.	Postman devolvería el código de error, y en la terminal del backend aparecería la petición fallida.
H2	La petición nunca se dispara, porque nadie llama a la función de carga.	Reviso si el ViewModel tiene un bloque <code>init { }</code> o si la pantalla tiene un <code>LaunchedEffect</code> .	En Logcat no aparecería ninguna petición al abrir la pantalla. Ese silencio sería la prueba.
H3	El <code>data class</code> no coincide con los nombres del JSON, así que la lista se deserializa vacía.	Comparo el JSON real que devuelve Postman contra las anotaciones <code>@SerializedName</code> del modelo.	La petición saldría con código 200, pero los objetos llegarían con los campos en <code>null</code> o en cero.
H4	El estado del ViewModel se actualiza, pero la pantalla observa otra variable.	Reviso qué variable lee el Composable y si es la misma que asigna el ViewModel.	Los datos llegarían bien y hasta se podrían ver en Logcat, pero la pantalla seguiría vacía.
H5	Falta el token de autenticación y el servidor responde 401.	Miro si esa ruta del backend exige token, y si la app lo envía en el encabezado <code>Authorization</code> .	La respuesta sería 401 y en Logcat se vería la petición sin el encabezado <code>Authorization</code> .

Estas hipótesis quedan escritas **antes** de tocar el código, tal como pide la guía. En la actividad PM1-AA3-EV05 se compara cuál se confirmó y cuál se descartó, y con qué evidencia.

3. ¿Qué es un «estado» en Jetpack Compose?

Un estado es un valor que la interfaz observa. Cuando ese valor cambia, Compose vuelve a dibujar automáticamente las partes de la pantalla que dependían de él. A ese redibujado se le llama **recomposición**.

La diferencia con el Android clásico es de responsabilidad: antes el programador cambiaba la vista a mano (`texto.text = "..."`); en Compose el programador sólo describe cómo debe verse la pantalla según los datos, y el framework se encarga de actualizarla.

¿Por qué una pantalla puede quedarse vacía aunque los datos sí hayan llegado?

Porque «que lleguen» y «que la pantalla los observe» son dos cosas distintas. Tres casos típicos, todos reales:

CASO	QUÉ PASA
El dato se guarda en una variable normal	Una variable común no avisa a nadie cuando cambia. Sólo <code>mutableStateOf</code> crea un valor observable.
El dato se guarda en un estado distinto del que lee la pantalla	El ViewModel llena una variable y el Composable lee otra: los dos tienen razón y la pantalla queda vacía.
La lista se modifica en vez de reemplazarse	Con una lista normal, agregar elementos no cambia la referencia, así que Compose no se entera. Para eso existe <code>mutableStateListOf</code> .

En los tres casos el dato está en memoria, el servidor respondió bien y no hay ningún error. Simplemente nadie le avisó a la pantalla.

4. ¿Qué diferencia hay entre «el código compila» y «el código funciona»?

Compilar significa que el código está bien escrito según las reglas del lenguaje: los tipos concuerdan, los nombres existen, la sintaxis es válida. **Funcionar** significa que, al ejecutarse y hablar con el resto del mundo, produce el resultado esperado. Son dos cosas independientes.

¿Por qué un error de integración no lo detecta el compilador?

Porque el compilador sólo puede revisar lo que tiene delante: el código de este proyecto. No puede saber qué va a responder el servidor, porque eso lo decide otro programa, escrito en otro lenguaje, que corre en otra máquina y que puede cambiar mañana sin que este código se entere.

El ejemplo exacto de este caso: si en Kotlin se declara que una petición devuelve una lista de países, el compilador acepta esa declaración sin objetar, porque es una frase válida. Que el servidor realmente devuelva una lista, o un objeto que contiene la lista, es algo que sólo se descubre en tiempo de ejecución, cuando Gson intenta traducir el JSON y no encuentra la forma que esperaba.

Dicho de otro modo: **el compilador verifica el contrato que yo escribí, no el contrato que el servidor cumple.** Cuando esos dos contratos no coinciden, el proyecto compila igual y falla al ejecutarse. Eso es un error de integración, y por eso hacen falta el Logcat y las pruebas con un cliente HTTP: son las únicas herramientas que ven los dos lados a la vez.